EVENT REGISTRATION SYSTEM

CPP10 Project Report

Student Name: [ENTER YOUR NAME]

Enrollment Number: [ENTER YOUR ENROLLMENT NUMBER]

Course / Class: [ENTER COURSE / CLASS]

Institution: [ENTER INSTITUTION NAME]

GitHub Repository: [ADD YOUR REPOSITORY URL]

Academic Session: 2026–2027

1. Abstract

The Event Registration System is a menu-driven C++ application designed to maintain event, participant, organizer, and registration records. It supports event creation, participant and organizer management, registration and cancellation, and persistent storage through text files. The system applies object-oriented programming, modular design, validation, exception handling, and standard-library containers.

2. Introduction

Manual event registration can lead to duplicate entries, incorrect seat counts, and scattered records. This project centralizes basic event information and registration operations in a single application. Its main goal is to demonstrate a small but practical information-management system using C++.

3. Problem Statement

A simple event organizer needs a reliable way to list events, record participants, prevent duplicate registrations, enforce capacity limits, cancel registrations, and retain information after the application closes.

4. Objectives

• Create and display event records.

• Store participant and organizer details.

• Register participants for events and reject duplicates.

• Enforce event capacity and update seat counts.

• Cancel registrations and release seats.

• Store data in files so it can be loaded in later runs.

• Apply OOP, modular programming, validation, and exception handling.

5. Scope

The application is a local, single-user console program. It is suitable as a learning prototype and small demonstration. It does not provide network access, authentication, payment processing, concurrent users, or a graphical interface.

6. Tools and Technologies

• Language: C++ (C++11 or later)

• Standard library: string, vector, fstream, algorithm, iostream, exception, ctime

• Storage: pipe-delimited text files

• UML: PlantUML source notation

• Build: g++ or a compatible C++ compiler

7. System Requirements

Hardware: A standard desktop/laptop capable of running a C++ compiler.

Software: Windows, Linux, or macOS; a C++11-compatible compiler; terminal/command prompt.

8. Functional Requirements

1. List available events with capacity and remaining seats.

2. Add events with name, date, venue, and positive capacity.

3. Add participants with a valid-looking email address and unique email.

4. List participants.

5. Add and list organizers.

6. Register a participant for an event.

7. Reject invalid IDs, duplicate registrations, and full events.

8. List and cancel registrations.

9. Save records to text files and load them on startup.

9. OOP Design

• **Event:** Holds event details and manages registration count and remaining seats.

• **Participant:** Holds participant identity and contact details.

• **Organizer:** Holds organizer and department/team details.

• **Registration:** Links a participant to an event and stores a timestamp.

• **EventSystem:** Coordinates menu operations, validation, and business rules.

• **FileManager:** Loads and saves records to the Data folder.

Encapsulation is used by keeping fields private and exposing public methods. Vectors hold collections of objects. Exceptions report invalid operations, including a full event or invalid input. Source code is divided into header and implementation files.

10. Main Transaction / Process

The main transaction is participant event registration:

1. The user selects registration from the menu.

2. The application asks for participant ID and event ID.

3. It checks that both records exist.

4. It rejects a registration if the same participant is already registered for the same event.

5. The Event object checks remaining capacity; a full event is rejected.

6. A Registration record is created and the event's registered count is incremented.

7. The event, participant, organizer, and registration records are saved.

8. A confirmation and registration ID are displayed.

Cancellation performs the reverse seat-count adjustment and deletes the registration record.

11. Data Design

File Fields

--- ---

events.txt id, name, date, venue, capacity, registered

participants.txt id, name, email

organizers.txt id, name, department

registrations.txt registration ID, participant ID, event ID, timestamp

Each record uses a pipe (|) delimiter. Field values must not contain the delimiter. The files are plain text and can be inspected with a text editor.

12. UML Diagrams

Four PlantUML source files are included in the Diagrams folder:

• `class_diagram.puml` — classes, attributes, and relationships.

• `use_case_diagram.puml` — system functions and external roles.

• `activity_diagram.puml` — registration decision flow.

• `sequence_diagram.puml` — interaction between user, EventSystem, Event, Registration, and FileManager.

Render these sources with a PlantUML-compatible renderer if image versions are required by the instructor.

13. Testing and Expected Results

A manual test plan is provided in Tests/test_plan.txt. It covers normal operations, invalid input, duplicate registrations, capacity limits, cancellation, and persistence after restart. Expected behaviour is described in the test plan; actual execution results and screenshots must be captured on the target computer rather than assumed.

14. Limitations and Future Enhancements

Current limitations include simple email validation, local text-file storage, single-user operation, no login/roles, and no date-format validation beyond requiring a non-empty date. Future work could add a database, authentication, stronger validation, event search, report export, GUI/web interface, and automated unit tests.

15. Conclusion

The Event Registration System demonstrates how object-oriented C++ can be used to solve a practical record-management problem. It separates responsibilities across classes, validates key registration rules, prevents duplicate registrations, enforces event capacity, and stores data persistently. The project can be extended with a database, stronger security, and a graphical interface.

Appendix A: Build and Run

From the project root, compile:

g++ -std=c++11 Source/main.cpp Source/Event.cpp Source/Participant.cpp Source/Organizer.cpp Source/Registration.cpp Source/FileManager.cpp Source/EventSystem.cpp -o Executable/Project

Run from the Executable directory so the relative data path ../Data resolves:

cd Executable

./Project

On Windows with MinGW, use Project.exe.

Appendix B: Submission Completion

Before final upload, replace all bracketed identity fields, add the actual repository URL, compile and test on the college computer, capture genuine screenshots, render diagram images if required, and verify the institution's submission checklist.